

Évaluation du potentiel photovoltaïque des toitures

Documentation technique complète de la pipeline

Stage M1 – Université de Rennes

Juin 2026

Résumé

Ce document décrit en détail la pipeline de calcul du potentiel photovoltaïque (PV) des toitures développée pendant le stage. Il présente brièvement le but du projet et l'architecture générale, puis détaille *chaque fonction* du code : ce qu'elle fait, comment elle est construite, pourquoi ces choix, et les concepts (géomatiques, solaires, numériques) nécessaires pour la comprendre. Il se termine par la liste de ce qu'il reste à développer. La méthode suit l'étude suisse de Buffat *et al.* (2018), *A scalable method for estimating rooftop solar irradiation potential over large regions*, Applied Energy 216. Le document est écrit pour un lecteur qui ne connaît ni les bibliothèques utilisées (rasterio, geopandas, pvlib...) ni les données (LiDAR HD, BD TOPO, PVGIS), mais qui veut pouvoir lire et modifier le code.

Table des matières

1	Le projet en un coup d'œil	3
1.1	Objectif	3
1.2	La méthode de référence : Buffat et al. (2018)	3
1.3	État d'avancement	3
2	Architecture générale	3
2.1	Les deux branches de la pipeline	3
2.2	Arborescence et exécution	4
3	Les données d'entrée	5
3.1	LiDAR HD IGN : MNS et MNT	5
3.2	BD TOPO : les emprises de bâtiments	5
3.3	PVGIS et SARA3 : le rayonnement solaire	5
3.4	Systèmes de coordonnées	6
4	Notions de rayonnement solaire	6
4.1	Les composantes du rayonnement	6
4.2	La position du soleil	7
4.3	La transposition : du plan horizontal au plan du toit	7
4.4	L'ombrage par l'horizon	8
4.5	De l'irradiance à l'irradiation : le profil (mois × heure)	8
5	Branche B – la géométrie : détection des toitures	9
5.1	main_MNTMNS.py : l'orchestrateur	9
5.2	src/geometrie/ExtractBDtopo.py	9
5.2.1	tileBounds(mns_name) : du nom de fichier à l'emprise	9
5.2.2	loadBuild(gpkg_path, tile_bounds) : charger et filtrer les bâtiments	9
5.3	src/geometrie/clip_MNTMNS_pixel.py : clip_MNSMNT_pixels	10
5.3.1	Lecture des rasters et notion de transform affine	10

5.3.2	Le MNH et la rasterisation des bâtiments	10
5.3.3	Le pré-masque de validité	11
5.3.4	Pente et aspect par différences finies	11
5.3.5	L'érosion morphologique : nettoyer les bords	11
5.3.6	Les masques finaux	12
5.3.7	Les exports de contrôle	12
5.4	<code>src/geometrie/horizon.py</code> : <code>compHZ</code>	12
5.4.1	Le principe : un lancer de rayons discret	12
5.4.2	La vectorisation : la clé des performances	12
6	Branche A – la météo : tables d'irradiance par cellule	13
6.1	<code>src/irradiance/irr_fct.py</code> : le module central	13
6.1.1	Les constantes	14
6.1.2	<code>telecharger(lat, lon)</code> : la requête PVGIS	14
6.1.3	<code>transpAgr(bhi, dhi, lat, lon)</code> : le cœur du calcul	14
6.1.4	<code>profMH(serie, cles)</code> : la réduction (mois, heure)	16
6.1.5	<code>totAb(B, D)</code> : le contrôle annuel	16
6.1.6	<code>cheminTable</code> et <code>chargerTable</code> : le stockage	16
6.2	<code>src/irradiance/irr_dep.py</code> : le script de campagne	17
6.2.1	<code>grilleCellules()</code>	17
6.2.2	<code>construireCellule(lat, lon)</code>	17
6.2.3	<code>main()</code> : la boucle robuste	17
6.3	<code>src/irradiance/irr_tuille.py</code> : le pont dalle → cellule (helper)	18
7	La phase tuile : du pixel au GeoPackage	18
7.1	L'idée : un profil par pixel, masqué, puis intégré	18
7.2	<code>irrPixels</code> : profil, ombrage et énergie de chaque pixel	19
7.3	<code>masquerHorizon</code> : l'ombrage du direct	20
7.4	<code>agregerBatiment</code> : sommer par bâtiment	20
7.5	Du gisement à la production : le modèle PV simple	21
7.6	<code>irradianceTuile</code> : l'orchestrateur	22
7.7	Le GeoPackage de sortie	22
8	Conventions, unités et pièges récapitulés	23
9	Ce qu'il reste à faire	23
9.1	Ombrage : ce qui est capté, ce qui ne l'est pas	23
9.2	Interpolation bilinéaire (α, β)	24
9.3	Profil mensuels et toits plats	24
9.4	Modèle PV complet (raffinement)	24
9.5	Passage à l'échelle : multi-dalles et fusion par région	25
9.6	Validation	25
9.7	Limites connues et choix à réévaluer	25
	Références	26

1 Le projet en un coup d’œil

1.1 Objectif

Le but est de construire un outil capable d’estimer, **toiture par toiture**, la quantité d’énergie solaire annuelle (et mensuelle) qu’un toit reçoit, puis à terme l’électricité qu’une installation photovoltaïque y produirait. L’outil doit fonctionner **à grande échelle** : un bâtiment, une commune, un département, et idéalement la France entière. Cette contrainte d’échelle conditionne toute l’architecture : on ne peut pas se permettre des calculs lourds par bâtiment, il faut pré-calculer tout ce qui peut l’être et ne faire au niveau du pixel de toiture que des opérations très simples.

1.2 La méthode de référence : Buffat et al. (2018)

L’étude suisse de Buffat *et al.* a montré qu’on peut estimer le potentiel solaire de plus de 3 millions de bâtiments (toute la Suisse, à 0,5 m de résolution) avec une bonne précision (validation sur 500 installations réelles : $R^2 = 0,9$ sur la production mensuelle). Son idée centrale, que nous reprenons, est de **séparer le problème en deux échelles** :

1. **La météo varie à l’échelle de quelques kilomètres.** Le rayonnement solaire incident (nuages, turbidité) est fourni par des données satellites sur une grille d’environ 4–5 km. Pour chaque cellule de cette grille, on pré-calculé une *table* : l’irradiance moyenne reçue par un plan incliné, pour toutes les combinaisons d’orientation et de pente de toit, pour chaque mois et chaque heure de la journée.
2. **La géométrie varie à l’échelle de 50 cm.** La forme des toits (pente, orientation) et les ombres portées (relief, bâtiments voisins) sont extraites de modèles numériques d’élévation à 0,5 m issus du LiDAR. Pour chaque pixel de toiture, il suffit alors de *lire* dans la table pré-calculée la valeur correspondant à sa pente et son orientation, et de supprimer le rayonnement direct aux heures où le soleil est caché par l’horizon local.

Cette factorisation est ce qui rend la méthode scalable : le calcul coûteux (transposition du rayonnement, sections 4.3 et 6.1.3) est fait **une fois par cellule météo de $0,05^\circ$** (environ $4 \times 5,5$ km en France) au lieu d’une fois par pixel de 0,5 m – soit un facteur d’économie d’environ 10^8 .

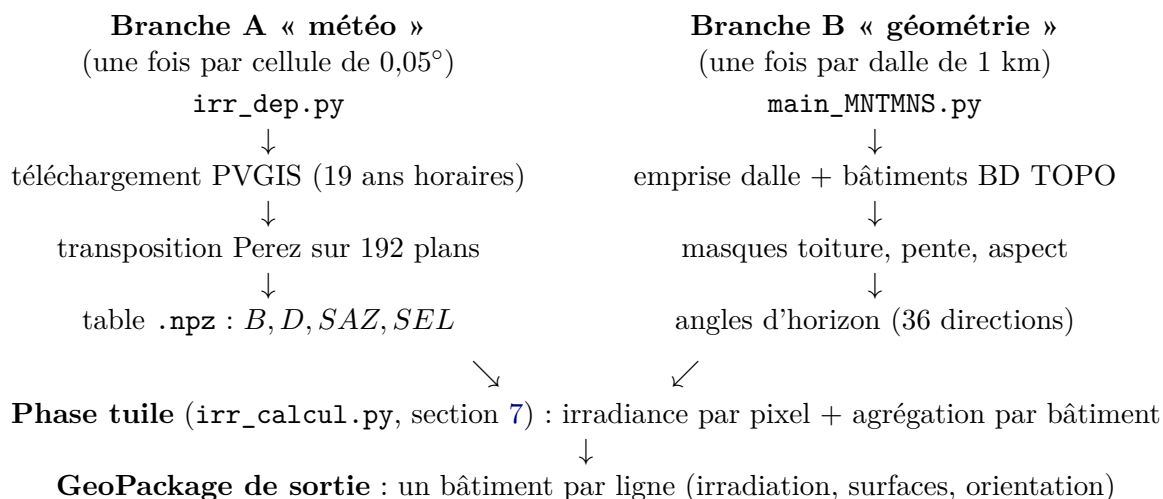
1.3 État d’avancement

- **Fait** : détection des pixels de toiture exploitables (croisement LiDAR / BD TOPO), calcul de la pente et de l’orientation de chaque pixel, calcul des angles d’horizon, construction des tables d’irradiance par cellule météo (téléchargement PVGIS + transposition Perez), et la **phase tuile** (section 7) : croisement des deux branches, irradiation par pixel **avec ombrage par l’horizon**, agrégation par bâtiment, **estimation de la production PV** (modèle simple), et écriture du **GeoPackage de sortie**. La pipeline tourne de bout en bout pour une dalle et produit un fichier bâtiments exploitable.
- **À faire** (section 9) : étendre l’horizon aux dalles voisines (effet de bord), raffiner l’interpolation (α, β), le modèle PV complet, et le passage à l’échelle (boucle multi-dalles + fusion par région).

2 Architecture générale

2.1 Les deux branches de la pipeline

La pipeline travaille par **dalle** (tuile) de $1 \text{ km} \times 1 \text{ km}$, le format de distribution des données LiDAR de l’IGN. Deux branches indépendantes préparent les ingrédients, une phase finale les croise :



L'angle d'horizon (branche B) est calculé puis *utilisé* par la phase tuile pour l'ombrage du rayonnement direct. La seule limite actuelle est que l'horizon est calculé sur la dalle seule (sans ses voisines), donc sous-estimé en bordure de dalle (section 9).

Les deux branches ne communiquent que par **des fichiers** : des tables `.npz` d'un côté, des GeoTIFF de l'autre. C'est volontaire : chaque étape peut être relancée, vérifiée ou parallélisée indépendamment, et un plantage ne fait jamais perdre que l'étape en cours.

2.2 Arborescence et exécution

```

Evaluation_PV_toitures_France/
|-- main_MNTMNS.py          # orchestre UNE dalle (geometrie +
    irradiance), pour tests
|-- src/
|   |-- geometrie/          # branche geometrie (par dalle)
|   |   |-- ExtractBDtopo.py # emprise de dalle + chargement BD TOPO
|   |   |-- clip_MNTMNS_pixel.py # masques toiture + pente + aspect
|   |   |-- horizon.py       # angles d'horizon par raycast
|   |-- irradiance/
|   |   |-- irr_fct.py       # moteur lookup table (transposition +
|   |   | lecture)
|   |   |-- irr_dep.py       # campagne : construit la lookup table
|   |   |-- irr_calcul.py    # lookup table -> irradiance + production
|   |   | par batiment
|   |   |-- irr_tuile.py     # helper coords/centre d'une dalle (tests,
|   |   | hors pipeline finale)
|-- data/
|   |-- raw/                # donnees brutes (MNS, MNT, BD TOPO .gpkg)
|   |-- tables/             # tables meteo .npz produites par irr_dep
|   |-- processed/          # sorties par dalle (masques, horizons, .
|   |   gpkg)
|-- environment.yml          # environnement conda "stage-lidar"
  
```

Tout se lance **depuis la racine du projet**, dans l'environnement conda stage-lidar :

```

python main_MNTMNS.py          # une dalle complete (geometrie +
    irradiance)
python src/irradiance/irr_dep.py # branche A (meteo, par bbox)
  
```

Les chemins relatifs (`data/...`) sont résolus par rapport au répertoire courant, d'où l'importance de lancer depuis la racine.

3 Les données d'entrée

3.1 LiDAR HD IGN : MNS et MNT

Le programme **LiDAR HD** de l'IGN couvre la France d'un nuage de points laser aéroporté dense (≥ 10 points/m²). L'IGN en dérive deux *modèles numériques d'élévation*, distribués en dalles GeoTIFF de 1 km × 1 km à 0,5 m de résolution (rasters de 2000 × 2000 pixels) :

MNS (Modèle Numérique de Surface)

altitude du *sursol* : on y voit les toits, les arbres, tout ce qui dépasse. C'est lui qui porte la géométrie des toitures.

MNT (Modèle Numérique de Terrain)

altitude du *sol nu*, comme si tous les objets étaient effacés.

La différence MNH = MNS – MNT (modèle numérique de *hauteur*) donne la hauteur des objets au-dessus du sol : ≈ 0 sur le sol, plusieurs mètres sur un toit ou un arbre. C'est le critère central de la détection (section 5.3).

Convention de nommage des dalles. Un fichier s'appelle par exemple LHD_FXX_0495_6611_MNS_0_0M50_LAMB93_IGN69.tif. Les deux nombres 0495 et 6611 sont les coordonnées Lambert 93 en km du coin nord-ouest de la dalle (vérifié sur les données : pour la dalle 0475_6594, l'emprise lue dans le GeoTIFF va de $y = 6\,593\,000$ à $y = 6\,594\,000$ m). Attention, c'est une source d'erreur classique : le nom donne le coin *haut*-gauche, pas le coin bas-gauche.

Valeur nodata. Les pixels sans donnée valent –9999 dans les fichiers IGN. Partout dans le code, ils sont convertis en NaN (*Not a Number*) dès la lecture, pour qu'aucun calcul ne les prenne pour une altitude réelle de –9999 m.

3.2 BD TOPO : les emprises de bâtiments

La **BD TOPO** est la base vectorielle de référence de l'IGN. On utilise sa couche **batiment** : un polygone par bâtiment (l'emprise au sol, précision métrique), avec des attributs. Elle est distribuée par département au format **GeoPackage** (.gpkg), un conteneur SQLite standard pour données géographiques. Les attributs conservés par la pipeline :

cleabs	identifiant unique IGN du bâtiment (clé de jointure finale)
nature, usage_1	typologie (église, serre, industriel...)
hauteur, nombre_d_etages	contrôles de cohérence possibles
etat_de_l_objet	on ne garde que « En service »
construction_legere	on exclut les structures légères (auvents...)

Le rôle de la BD TOPO est de dire *où chercher des toits* : sans elle, le MNH seul ne distingue pas un toit d'un arbre.

3.3 PVGIS et SARAH-3 : le rayonnement solaire

PVGIS (Photovoltaic Geographical Information System) est un service web gratuit du Centre commun de recherche de la Commission européenne (JRC). Son API (version 5.3, https://re.jrc.ec.europa.eu/api/v5_3/) fournit des séries temporelles **horaires** de rayonnement au sol, issues de la base **SARAH-3**¹ : des estimations dérivées des images du satellite géostationnaire Météosat, sur une grille de 0,05° × 0,05° (environ 3,8 × 5,5 km à la latitude de la France), disponibles de 2005 à 2023.

1. SARAH-3 (CM SAF / EUMETSAT) : rayonnement satellite à 0,05°, pas 30 min, depuis 1983 – voir Réf. PVGIS n'en utilise ici qu'un sous-ensemble horaire 2005–2023.

C'est l'équivalent (en plus récent) des données « CM SAF / Heliosat SARAH » utilisées par Buffat. L'avantage du satellite sur les stations météo au sol : une couverture spatiale continue et homogène sur 19 ans, là où les stations sont éparées et demanderaient une interpolation hasardeuse.

Pour chaque heure, PVGIS fournit (entre autres) les deux composantes du rayonnement sur un plan horizontal – direct et diffus – définies en section 4.1. La requête est faite avec une inclinaison nulle précisément pour récupérer ces grandeurs horizontales de base.

3.4 Systèmes de coordonnées

Deux systèmes cohabitent, et la pipeline passe de l'un à l'autre explicitement :

Lambert 93 (code EPSG 2154)

la projection plane officielle de la France métropolitaine. Coordonnées (x, y) **en mètres** ; toutes les données IGN (LiDAR, BD TOPO) sont dans ce système. On peut y mesurer des distances et des surfaces directement.

WGS 84 (code EPSG 4326)

latitude/longitude **en degrés**, le système GPS. C'est ce qu'attend l'API PVGIS et ce qu'utilisent les calculs astronomiques de position du soleil.

La conversion est faite par la bibliothèque `pyproj` (section 6.3). Détail important : l'option `always_xy=True` impose l'ordre (longitude, latitude) – sans elle, `pyproj` suit l'ordre officiel du système, qui est (latitude, longitude) pour le WGS84, source classique d'inversions silencieuses.

4 Notions de rayonnement solaire

Cette section pose les concepts physiques utilisés par la branche météo. Tout le vocabulaire défini ici est réutilisé dans la description des fonctions.

4.1 Les composantes du rayonnement

L'**irradiance** est une puissance surfacique instantanée, en W/m^2 . L'**irradiation** est son intégrale dans le temps, en Wh/m^2 (ou kWh/m^2). Au sol, le rayonnement reçu par une surface se décompose en trois parts d'origines différentes :

Direct (*beam*)

les rayons qui arrivent en ligne droite du disque solaire. C'est la part qui crée les ombres, et donc la seule affectée par les masques (relief, bâtiments voisins).

Diffus

la lumière du ciel : rayonnement diffusé par les molécules, les aérosols et les nuages, qui arrive de toute la voûte céleste. Par temps couvert, c'est la totalité du rayonnement.

Réfléchi

la part renvoyée par le sol environnant vers une surface inclinée. Elle est proportionnelle à l'**albédo** ρ du sol (part réfléchie, sans dimension) ; on prend $\rho = 0,2$, valeur standard pour un environnement urbain/végétal moyen (même valeur que Buffat).

Sur un **plan horizontal**, on note (conventions internationales, reprises par `pvlb`) :

- *GHI* (*Global Horizontal Irradiance*) : le total horizontal ;
- *BHI* (*Beam Horizontal Irradiance*) : la part directe ;
- *DHI* (*Diffuse Horizontal Irradiance*) : la part diffuse ;

avec la relation de fermeture $GHI = BHI + DHI$ (le réfléchi est nul sur un plan horizontal, qui ne « voit » pas le sol).

Enfin, le **DNI** (*Direct Normal Irradiance*) est l'irradiance directe mesurée sur un plan *perpendiculaire aux rayons*. C'est la grandeur géométriquement « pure » du faisceau : connaître le DNI permet de projeter le direct sur n'importe quel plan. Lien avec le direct horizontal :

$$DNI = \frac{BHI}{\cos \theta_z}, \quad (1)$$

où θ_z est l'angle zénithal du soleil (section 4.2). Cette division devient numériquement explosive quand le soleil est à l'horizon ($\cos \theta_z \rightarrow 0$) : c'est pour cela que le code passe par une fonction `pvlib` munie de garde-fous plutôt que par une division naïve (section 6.1.3).

4.2 La position du soleil

La position du soleil dans le ciel, vue d'un point donné à un instant donné, est décrite par deux angles :

- l'**azimut** γ_s : la direction horizontale, en convention boussole (0° = nord, 90° = est, 180° = sud, 270° = ouest) ;
- l'**élévation** θ_h (hauteur au-dessus de l'horizon) ou, de façon équivalente, l'**angle zénithal** $\theta_z = 90^\circ - \theta_h$ (écart à la verticale).

On utilise l'élévation *apparente* : l'atmosphère réfracte la lumière et relève légèrement le soleil visible par rapport au soleil géométrique (environ $0,5^\circ$ à l'horizon) – c'est la bonne grandeur à comparer à un horizon physique.

Le calcul est fait par l'algorithme **SPA** (*Solar Position Algorithm*, NREL), précis à $\pm 0,0003^\circ$, via `pvlib.solarposition.get_solarposition`. C'est exactement l'algorithme employé par Buffat. Toutes les heures de la pipeline sont en **UTC** (jamais en heure légale française) : les séries PVGIS sont en UTC, et garder un fuseau unique évite les pièges de l'heure d'été.

4.3 La transposition : du plan horizontal au plan du toit

Les données météo donnent le rayonnement sur un plan *horizontal* ; un toit est un plan *incliné*, caractérisé par sa pente β (0° = plat, 90° = vertical) et son orientation (azimut) α (convention boussole : $\alpha = 180^\circ$ pour un toit plein sud). Passer de l'un à l'autre s'appelle la **transposition**. Chaque composante se transpose différemment (équations 1 à 4 de Buffat) :

Le direct : pure géométrie. Il suffit de projeter le faisceau sur le plan :

$$G_{B,\alpha,\beta} = DNI \cdot \max(0, \cos \theta_i) = \frac{BHI}{\cos \theta_z} \cos \theta_i, \quad (2)$$

où θ_i est l'**angle d'incidence**, l'angle entre les rayons et la normale au plan :

$$\cos \theta_i = \cos \theta_z \cos \beta + \sin \theta_z \sin \beta \cos(\gamma_s - \alpha). \quad (3)$$

Le $\max(0, \cdot)$ traduit le fait qu'un soleil derrière le plan ne l'éclaire pas.

Le diffus : le modèle de Perez. Si le ciel était une voûte de luminance uniforme (modèle « isotrope »), un plan de pente β en verrait la fraction $(1 + \cos \beta)/2$. En réalité le ciel est plus brillant autour du soleil (composante *circumsolaire*) et près de l'horizon (*bande d'horizon*). Le modèle empirique de **Perez et al. (1990)** corrige ces deux effets :

$$G_{D,\alpha,\beta} = DHI \left[(1 - F_1) \frac{1 + \cos \beta}{2} + F_1 \frac{a}{b} + F_2 \sin \beta \right], \quad (4)$$

où F_1 et F_2 sont des coefficients empiriques (poids du circumsolaire et de la bande d'horizon) qui dépendent de l'état du ciel – décrit par un indice de clarté et une luminosité calculés à

partir de DHI , DNI , de la masse d'air et du rayonnement extraterrestre – et a/b est un terme géométrique de projection du circumsolaire ($a = \max(0, \cos \theta_i)$, $b = \max(\cos 85^\circ, \cos \theta_z)$). C'est le modèle retenu par Buffat (et par la communauté PV en général) car il est nettement plus juste que l'isotrope, notamment pour des plans verticaux ou orientés est/ouest. Il est implémenté dans `pvlib`; le code ne fait que l'appeler avec les bons ingrédients.

Le réfléchi : modèle isotrope au sol. Le plan incliné « voit » le sol avec le facteur de vue complémentaire $(1 - \cos \beta)/2$, et le sol renvoie $\rho \cdot GHI$:

$$G_{R,\alpha,\beta} = \rho \cdot GHI \frac{1 - \cos \beta}{2}. \quad (5)$$

Total dans le plan. La somme des trois (équation 4 de Buffat) :

$$G_{\alpha,\beta} = G_{B,\alpha,\beta} + G_{D,\alpha,\beta} + G_{R,\alpha,\beta}. \quad (6)$$

Dans le code, le diffus ciel (Perez) et le réfléchi sont regroupés dans un même tableau D (« tout ce qui n'est pas du direct »), car c'est la part qui *subsiste* quand le soleil est masqué; le direct B est gardé séparé car c'est lui qu'on annulera à l'ombre.

4.4 L'ombrage par l'horizon

Depuis un pixel de toit, dans une direction donnée, l'**angle d'horizon** $\theta_{hor}(\varphi)$ est l'élévation angulaire du plus haut obstacle visible (faîtage voisin, immeuble, colline). La règle d'ombrage de Buffat (équations 5 et 6) est binaire et ne s'applique qu'au direct :

$$\text{si } \theta_h \leq \theta_{hor}(\gamma_s) \implies G_B = 0 \quad (\text{le diffus et le réfléchi sont conservés}). \quad (7)$$

Autrement dit : à chaque pas de temps, on regarde l'azimut du soleil, on lit l'angle d'horizon dans cette direction, et si le soleil est plus bas que l'obstacle, le pixel est à l'ombre – il ne reçoit que du diffus. C'est une approximation (le diffus aussi est partiellement masqué par les obstacles), mais c'est celle de l'étude de référence et elle capte l'essentiel du phénomène, le direct représentant la majorité de l'énergie annuelle en France.

4.5 De l'irradiance à l'irradiation : le profil (mois $\times$ heure)

Plutôt que de garder 19 ans de séries horaires partout, la pipeline résume le climat par un **profil moyen (mois, heure)** : un tableau 12×24 dont la case (m, h) contient la moyenne, sur les 19 années, de l'irradiance aux heures h UTC du mois m (chaque case moyenne donc environ $19 \times 30 \approx 570$ valeurs horaires). C'est l'équivalent du « jour type mensuel » de Buffat (qui travaille à la demi-heure; PVGIS fournit de l'horaire).

L'énergie s'en déduit exactement comme dans l'équation 7 de Buffat : chaque case vaut pour 1 heure du jour type, donc

$$I_m = n_{\text{jours}}(m) \sum_{h=0}^{23} \bar{G}(m, h) \times 1 \text{ h} \quad [\text{Wh/m}^2/\text{mois}], \quad I_{\text{an}} = \frac{1}{1000} \sum_{m=1}^{12} I_m \quad [\text{kWh/m}^2/\text{an}]. \quad (8)$$

Pourquoi transposer *avant* de moyenner ? C'est un point méthodologique important (et le cœur de `transpAgr`). La transposition dépend de la position du soleil, qui change à chaque instant; transposer une moyenne reviendrait à éclairer le plan avec un « soleil moyen », ce qui fausse les géométries (notamment matin/soir et hiver). On transpose donc **chaque heure des 19 ans** avec sa vraie position de soleil, et on ne moyenne qu'ensuite. C'est exactement l'ordre des opérations de Buffat (« *for each of the available half-hour records [...] the irradiance values on inclined surfaces are computed* », puis agrégation par mois).

5 Branche B – la géométrie : détection des toitures

5.1 main_MNTMNS.py : l'orchestrateur

Ce script enchaîne toute la chaîne pour **une dalle**, configurée en tête de fichier (noms des fichiers MNS/MNT, chemin du GeoPackage BD TOPO, dossier de sortie). Son déroulé :

1. vérification de l'existence des fichiers d'entrée (sortie immédiate avec message clair sinon) ;
2. `tileBounds(MNS_NAME)` → emprise de la dalle en Lambert 93 ;
3. `loadBuild(GPKG_PATH, bounds)` → bâtiments BD TOPO de la dalle, filtrés ;
4. `clip_MNSMNT_pixels(...)` → masques de pixels de toiture (inclinés / plats), pente et aspect par pixel ;
5. `compHZ(...)` → angles d'horizon des pixels de toiture (36 directions de 10°), retournés en mémoire comme tableau ($N, 36$) et écrits en 36 rasters pour QGIS ;
6. `irradianceTuile(...)` (phase tuile, section 7) → irradiation par pixel *avec ombrage*, agrégation par bâtiment, et écriture du GeoPackage de sortie `<MNS>_irradiance.gpkg`.

Chaque étape est chronométrée et affiche des compteurs – utile pour estimer le coût au moment de passer à des milliers de dalles. Le script ne contient aucune logique métier : tout est dans les modules de `src/`, ce qui permet de réutiliser les fonctions ailleurs (notebooks, tests, futur traitement par lots).

5.2 src/geometrie/ExtractBDtopo.py

5.2.1 tileBounds(mns_name) : du nom de fichier à l'emprise

Rôle. Dédurre l'emprise géographique ($x_{\min}, y_{\min}, x_{\max}, y_{\max}$) d'une dalle, en mètres Lambert 93, à partir de son seul nom de fichier – sans ouvrir le raster.

Fonctionnement. Une expression régulière extrait les deux groupes de 4 chiffres encadrés de tirets bas :

```
match = re.search(r'_(\d{4})_(\d{4})_', mns_name)
x_km = int(match.group(1)); y_km = int(match.group(2))
x_min = x_km * 1000; x_max = x_min + 1000
y_max = y_km * 1000; y_min = y_max - 1000      # nom IGN = coin NORD-
ouest
```

L'expression `r'_(\d{4})_(\d{4})_'` se lit : « un tiret bas, puis exactement 4 chiffres (capturés), un tiret bas, 4 chiffres (capturés), un tiret bas ». Si le nom ne correspond pas, la fonction lève une erreur explicite plutôt que de continuer avec des coordonnées fausses. Le point délicat est la dernière ligne : le nom encodant le coin *nord-ouest* (section 3.1), *y* nommé est le *haut* de la dalle ; on retranche 1 km pour obtenir le bas.

5.2.2 loadBuild(gpkg_path, tile_bounds) : charger et filtrer les bâtiments

Rôle. Renvoyer un `GeoDataFrame` (le « tableau avec une colonne géométrie » de la bibliothèque `geopandas`) contenant les bâtiments exploitables de la dalle.

Fonctionnement.

```
gdf = gpd.read_file(gpkg_path, layer='batiment', bbox=tile_bounds)
gdf = gdf[(gdf['etat_de_l_objet'] == 'En service') &
          (gdf['construction_legere'] == False)].copy().reset_index(drop=
          True)
gdf = gdf[['cleabs', 'nature', 'usage_1', 'hauteur',
          'nombre_d_etages', 'geometry']].copy()
```

Trois idées :

- **Lecture spatiale partielle.** L'argument `bbox` fait utiliser l'index spatial du GeoPackage : seuls les bâtiments intersectant la dalle sont lus. Un département entier contient des centaines de milliers de bâtiments ; en lire ~ 1000 prend une fraction de seconde au lieu de plusieurs dizaines.
- **Filtrage attributaire.** On écarte les bâtiments hors service (ruines, en projet) et les constructions légères, dont les « toits » ne portent pas de PV. Le filtre est volontairement minimal à ce stade ; un filtrage par `nature/usage_1` (serres, monuments historiques...) est en commentaire dans le code, à activer plus tard.
- **`reset_index(drop=True)`.** Après filtrage, l'index du GeoDataFrame est renuméroté 0, 1, 2, ... : c'est cet index (décalé de +1, voir plus bas) qui servira d'identifiant de bâtiment dans les rasters, et il doit être dense et prévisible.

5.3 src/geometrie/clip_MNTMNS_pixel.py : clip_MNSMNT_pixels

Rôle. C'est la fonction de **détection** : elle décide, pixel par pixel (0,5 m), ce qui est une toiture exploitable, et calcule pour chacun sa pente et son orientation. Elle prend les chemins du MNS et du MNT, le GeoDataFrame des bâtiments, et deux seuils ; elle renvoie deux masques entiers (toits inclinés / toits plats), la pente, l'aspect, et les métadonnées raster nécessaires pour écrire les sorties.

5.3.1 Lecture des rasters et notion de transform affine

```
with rasterio.open(mns_path) as src:
    mns = src.read(1).astype(np.float32) # bande 1 -> tableau
        numpy (2000, 2000)
    transform = src.transform # lien indices <->
        coordonnees terrain
    meta = src.meta.copy() # dimensions, CRS,
        nodata, ...
    nodata = src.nodata
    resolution = src.res[0] # 0.5 m
```

Un GeoTIFF est une matrice de pixels accompagnée d'une **transformation affine** qui convertit les indices (colonne, ligne) en coordonnées terrain (x, y) : $x = a \cdot \text{col} + c$, $y = e \cdot \text{lig} + f$ avec $e < 0$ – la ligne 0 est en *haut* (au nord) et les lignes croissent vers le *sud*. Cette orientation « lignes vers le sud » est invisible tant qu'on fait des calculs locaux, mais devient cruciale pour l'aspect et pour l'horizon (sens des déplacements dans la matrice). Après lecture, les pixels `nodata` sont convertis en `NaN` :

```
if nodata is not None:
    mns[mns == nodata] = np.nan
    mnt[mnt == nodata] = np.nan
```

5.3.2 Le MNH et la rasterisation des bâtiments

```
mnh = mns - mnt # hauteur au-dessus du
sol

geometries = gdf.geometry.buffer(buffer) # buffer = 1.2 m
masque_bat = rasterize(
    zip(geometries, gdf.index + 1), # index + 1 : 0 = fond
    out_shape=mns.shape, transform=transform, fill=0, dtype="int32")
```

- **Le buffer de 1,2 m** dilate chaque polygone : les emprises BD TOPO sont les murs au sol, or les toits débordent (avant-toits), et il existe un léger décalage possible entre la BD et le LiDAR. Sans buffer, on perdrait systématiquement les bords de toits.
- **rasterize** « brûle » chaque polygone dans une grille alignée sur le MNS : chaque pixel dont le centre tombe dans le polygone i reçoit la valeur $i + 1$, les autres restent à `fill=0`. On obtient d'un coup *où* sont les bâtiments *et lequel* : le raster sert à la fois de masque et de carte d'identifiants, ce qui permettra d'agréger les résultats par bâtiment sans aucune boucle sur les polygones.
- **Pourquoi +1 ?** L'index du GeoDataFrame commence à 0, qui est aussi la valeur de fond : sans le décalage, le premier bâtiment de chaque dalle serait invisible (confondu avec le fond). Pour retrouver le bâtiment depuis un pixel : `gdf.loc[valeur - 1]`.

5.3.3 Le pré-masque de validité

```
pts_ok = ((masque_bat > 0) & (mnh >= mnh_min) &
          np.isfinite(mns) & np.isfinite(mnt))
```

Un pixel candidat doit (i) être dans une emprise (buffée) de bâtiment, (ii) être à au moins `mnh_min = 1,5 m` au-dessus du sol – ce qui élimine le sol nu, les terrasses basses, voitures et murets inclus dans le buffer – et (iii) avoir des altitudes valides dans les deux modèles. Le seuil de 1,5 m est un compromis : assez haut pour exclure le bruit au sol, assez bas pour garder les toits de plain-pied.

5.3.4 Pente et aspect par différences finies

```
dz_dy, dz_dx = np.gradient(mns, resolution)
pente = np.degrees(np.arctan(np.sqrt(dz_dx**2 + dz_dy**2)))
aspect = np.degrees(np.arctan2(-dz_dx, dz_dy)) % 360
```

`np.gradient` calcule les dérivées partielles de l'altitude par différences finies *centrées* : pour chaque pixel, la variation d'altitude entre ses deux voisins, divisée par la distance ($2 \times 0,5$ m). Le premier tableau renvoyé est la dérivée le long de l'axe 0 (les lignes), le second le long de l'axe 1 (les colonnes).

- **La pente** est l'angle dont le plan local s'écarte de l'horizontale : la norme du gradient $\|\nabla z\|$ est la montée par mètre parcouru, et $\beta = \arctan \|\nabla z\|$.
- **L'aspect** est la direction *vers laquelle le toit descend* (la direction que « regarde » le pan), en convention boussole. Avec l'axe des lignes orienté vers le sud (cf. `transform`), la direction de descente est $(-\partial z / \partial x_{\text{est}}, -\partial z / \partial y_{\text{nord}}) = (-dz_dx, +dz_dy)$, et l'angle boussole d'un vecteur $(v_{\text{est}}, v_{\text{nord}})$ est `atan2(v_est, v_nord)` – d'où la formule. Le `% 360` ramène le résultat de $[-180, 180]$ vers $[0, 360[$. Vérifié sur cas synthétique : un plan montant vers l'est donne bien un aspect de 270° (il regarde l'ouest).

5.3.5 L'érosion morphologique : nettoyer les bords

```
kernel = np.ones((3, 3), dtype=bool)
pts_ok_erode = binary_erosion(pts_ok, structure=kernel)
```

L'**érosion binaire** (morphologie mathématique, via `scipy`) ne garde un pixel `True` que si *tout* son voisinage 3×3 est `True` : elle retire une couronne d'un pixel au bord du masque. Pourquoi ? Parce que le gradient centré d'un pixel de bord de toit utilise un voisin *hors* du toit (la façade, le sol) : sa pente et son aspect sont contaminés (pentes de 80° en bord de pan, aspects aléatoires). Plutôt que de filtrer ces artefacts par des seuils fragiles, on supprime la source : tout pixel dont un voisin est invalide.

5.3.6 Les masques finaux

```
masque_incline = np.where(pts_ok_erode & (pente >= 10) & (pente <= 70) &
                           (aspect >= 90) & (aspect <= 270), masque_bat,
                           0)
masque_plat     = np.where(pts_ok_erode & (pente < 10), masque_bat,
                           0)
```

Deux familles, traitées différemment ensuite :

- **toits inclinés** (10–70°) : les panneaux suivent le pan ; la borne haute écarte les façades résiduelles et les artefacts. Le filtre $90 \leq \text{aspect} \leq 270$ ne conserve que les pans orientés de l’est à l’ouest en passant par le sud – choix méthodologique assumé (les pans nord produisent peu en France), à réévaluer si l’on veut un cadastre exhaustif (section 9.7) ;
- **toits plats** (< 10°) : les panneaux y seraient posés sur châssis avec une inclinaison choisie ; leur potentiel se calculera avec un (α, β) conventionnel plutôt qu’avec la géométrie du pixel.

Les masques portent l’identifiant du bâtiment (`masque_bat`, donc index +1) là où la condition est vraie, 0 ailleurs : ce sont à la fois des masques booléens et des cartes de correspondance pixel $\rightarrow$ bâtiment.

5.3.7 Les exports de contrôle

Si `debug_dir` est fourni, la fonction écrit cinq GeoTIFF (MNH masqué, pente filtrée, aspect, deux masques) compressés en LZW, à ouvrir dans QGIS pour contrôle visuel. Deux détails de robustesse : le dossier est créé si besoin, et la conversion NaN $\rightarrow$ nodata n’est appliquée qu’aux rasters flottants (`np.isnan` déclencherait une erreur sur les masques entiers, qui ne peuvent pas contenir de NaN).

5.4 src/geometrie/horizon.py : compHZ

Rôle. Pour chaque pixel de toiture et chaque direction azimutale (36 directions par pas de 10°), calculer l’angle d’horizon $\theta_{hor}(\varphi)$ défini en section 4.4 : l’élévation du plus haut obstacle visible dans cette direction, à partir du MNS. **Sortie principale** : un tableau $(N, 36)$ en mémoire (un angle d’horizon en degrés par pixel de toiture et par direction), *retourné* pour être utilisé directement par la phase tuile (section 7). En effet de bord, la fonction écrit aussi un GeoTIFF par direction (`horizon_000.tif ... horizon_350.tif`, –9999 hors toiture, LZW) pour visualisation dans QGIS. L’ordre des lignes du tableau suit `np.where(masque_toiture)` : c’est ce qui garantit que la ligne k du tableau horizon correspond au même pixel que la ligne k côté phase tuile.

5.4.1 Le principe : un lancer de rayons discret

Pour un pixel d’altitude z_0 et une direction φ , on avance pas à pas le long du rayon ($k = 1, 2, \dots, k_{\max}$ pixels) ; à chaque pas, l’obstacle potentiel situé à la distance $k\delta$ ($\delta = 0,5$ m) et à l’altitude z_k est vu sous l’angle

$$\theta_k = \text{atan2}(z_k - z_0, k\delta), \quad (9)$$

et l’angle d’horizon est le maximum rencontré : $\theta_{hor}(\varphi) = \max_k \theta_k$. Avec `max_distance_m` = 100 m (réglage actuel dans `main`), $k_{\max} = 200$ pas. L’angle est initialisé à -90° ; un environnement entièrement plus bas que le pixel donne un horizon négatif, ce qui est correct (un faîtage voit « sous » l’horizontale).

5.4.2 La vectorisation : la clé des performances

L’algorithme naïf ferait trois boucles imbriquées (pixels $\times$ directions $\times$ pas) : des milliards d’itérations Python, rédhibitoire. L’astuce est de ne boucler en Python que sur les directions

(36) et les pas (200), et de traiter **tous les pixels de toiture d'un coup** à chaque pas, par indexation numpy :

```
x, y = np.where(masque_toiture)          # lignes/colonnes des N pixels de
    toit
z0 = mns[x, y]                           # leurs N altitudes

for phi_deg in range(0, 360, step):
    dx = -np.cos(phi_rad)                 # nord = lignes qui diminuent
    dy = np.sin(phi_rad)                  # est = colonnes qui augmentent
    theta = np.full(len(x), -np.pi/2, dtype=np.float32)
    for k in range(1, max_dist_px + 1):
        x_k = np.clip(np.round(x + k*dx).astype(int), 0, H-1)
        y_k = np.clip(np.round(y + k*dy).astype(int), 0, W-1)
        angle = np.arctan2(mns[x_k, y_k] - z0, k * res)
        theta = np.fmax(theta, angle)     # max courant ; fmax ignore les
                                          NaN
```

Points à comprendre :

- **Conventions d'axes.** Dans la matrice, *x* désigne la *ligne* et *y* la *colonne* (ordre numpy). La ligne 0 étant au nord, aller vers le nord c'est décrémenter la ligne, d'où $dx = -\cos \varphi$; aller vers l'est c'est incrémenter la colonne, d'où $dy = +\sin \varphi$. Vérifié par test synthétique : un mur de 10 m placé 15 m au nord d'un pixel donne $\theta_{hor}(0^\circ) = \arctan(10/15) = 33,69^\circ$, exactement la valeur attendue.
- **Échantillonnage au plus proche voisin.** `np.round` arrondit la position réelle du rayon au pixel le plus proche – une forme discrète de tracé de ligne, suffisante à 0,5 m de résolution.
- **`np.clip` aux bords.** Quand le rayon sort de la dalle, on ré-échantillonne le pixel de bord : l'angle correspondant décroît avec la distance et n'affecte plus le maximum. Conséquence assumée : près des bords de dalle, l'horizon est sous-estimé (les obstacles de la dalle voisine sont invisibles); la correction (charger un voisinage) est listée dans le travail restant.
- **`np.fmax` et non `np.maximum`.** Les deux calculent le maximum élément par élément, mais `np.maximum` *propage* les NaN : un seul pixel nodata sur le rayon aurait définitivement contaminé l'angle. `np.fmax` ignore les NaN, ce qui revient à traiter les trous de données comme transparents.
- **Coût.** 36×200 opérations vectorisées sur *N* pixels de toiture : quelques secondes par dalle pour $N \sim 10^5$. La distance de recherche est le paramètre de coût principal (100 m actuellement; Buffat utilise 500 m pour l'horizon proche, complété d'un horizon lointain – voir section 9).

Pourquoi **36 directions** ? L'horizon varie lentement avec l'azimut ; au moment du masquage, on lira la direction la plus proche de l'azimut solaire (erreur maximale 5° , du même ordre que le pas de 5° utilisé par Buffat). Et pourquoi **un raster par direction** ? Pour rester en 2D simple (facile à ouvrir dans QGIS), au prix de 36 fichiers – compressés LZW, ils sont essentiellement vides hors toiture et pèsent peu.

6 Branche A – la météo : tables d'irradiance par cellule

6.1 `src/irradiance/irr_fct.py` : le module central

Ce module contient les constantes et toutes les fonctions réutilisables de la branche météo. Il est conçu pour être importable depuis n'importe où (`from src.irradiance.irr_fct import *` depuis la racine).

6.1.1 Les constantes

```
ALBEDO = 0.2 # albedo du sol (Buffat 2018)
ALPHAS = np.arange(0, 360, 15) # 24 orientations (0=Nord, 90=Est, ...)
BETAS = np.arange(0, 71, 10) # 8 pentes, 0 a 70 degres
N_JOURS = np.array([31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31])
PAS = 0.05 # pas de la grille meteo PVGIS (degres)
DOSSIER = "data/tables" # un .npz par cellule meteo
URL = "https://re.jrc.ec.europa.eu/api/v5_3/" # PVGIS 5.3 (SARAH-3)
```

La grille (α, β) est volontairement **grossière** ($15^\circ \times 10^\circ$, 192 combinaisons) : l'irradiance annuelle varie lentement et régulièrement avec l'orientation et la pente (c'est une surface lisse, cf. figure 2 de Buffat), donc une interpolation depuis cette grille suffira au niveau du pixel. Buffat utilise exactement ces pas (15° en orientation, 10° en pente). La borne de 70° couvre la gamme acceptée par le masque incliné ($10\text{--}70^\circ$) : aucune extrapolation ne sera nécessaire.

6.1.2 telecharger(lat, lon) : la requête PVGIS

Rôle. Récupérer la série horaire 2005–2023 des composantes du rayonnement horizontal pour la cellule SARAH-3 contenant le point demandé.

```
out = pvlib.iotools.get_pvgis_hourly(
    lat, lon, start=2005, end=2023,
    raddatabase="PVGIS-SARAH3",
    components=True, surface_tilt=0, surface_azimuth=0,
    usehorizon=False,
    url=URL, map_variables=True)
df = out[0]
```

Décodage des arguments :

- `components=True` demande les composantes séparées plutôt que le seul total; `surface_tilt=0` demande le plan *horizontal*, si bien que la composante « directe dans le plan » renvoyée (`poa_direct`) est le *BHI*, et la « diffuse ciel » (`poa_sky_diffuse`) est le *DHI* (section 4.1); la part réfléchie est nulle à plat.
- `usehorizon=False` : PVGIS sait appliquer l'ombrage du relief lointain à ses séries, mais nous gérons l'horizon nous-mêmes (et à bien plus fine échelle); on demande donc le rayonnement *non masqué* pour ne pas compter l'ombrage deux fois. Garder l'horizon lointain PVGIS de côté reste une option notée dans le code.
- `map_variables=True` renomme les colonnes PVGIS (`Gb(i)`, `Gd(i)`...) en noms standard `pvlib`.
- Le résultat `out` est un tuple dont le premier élément est le DataFrame (indexé par des horodatages UTC) : 19 ans $\times$ 8760 h $\approx$ 166 500 lignes.

6.1.3 transpAgr(bhi, dhi, lat, lon) : le cœur du calcul

Rôle. Transformer 19 ans de séries horaires *horizontales* en **tables de profils moyens sur plans inclinés** : pour chacune des 24×8 combinaisons (α, β) , le profil 12×24 (mois $\times$ heure UTC) de l'irradiance directe *B* et diffuse-plus-réfléchie *D*; plus les profils 12×24 de position moyenne du soleil (*SAZ* : azimut, *SEL* : élévation apparente). C'est l'implémentation de la phase 1 de Buffat (sections 4.3 et 4.5).

Étape 1 – préparation des ingrédients (une seule fois, hors boucle) :

```
times = bhi.index
ghi = (bhi + dhi).clip(lower=0) # relation de
    fermeture
```



```

sp = pvlib.solarposition.get_solarposition(times, lat, lon)    # SPA, ~166
                    500 instants
dni = pvlib.irradiance.dni(ghi, dhi, sp["apparent_zenith"]).fillna(0)
dni_extra = pvlib.irradiance.get_extra_radiation(times)
airmass = pvlib.atmosphere.get_relative_airmass(sp["apparent_zenith"])
cles = [times.month, times.hour]

```

- `sp` contient pour chaque heure le zénith apparent, l'azimut, l'élévation apparente du soleil (algorithme SPA, section 4.2), calculés au centre de la cellule météo – la position du soleil varie de façon négligeable à l'échelle d'une cellule de 5 km.
- `pvlib.irradiance.dni` reconstruit le *DNI* par l'équation (1), avec garde-fous : les valeurs négatives (bruit de mesure à l'aube) et les valeurs au-delà de 88° de zénith (division quasi nulle, donc instable) sont neutralisées (NaN puis 0 via `fillna`) ; c'est la raison pour laquelle on n'écrit pas la division à la main.
- `dni_extra` est le rayonnement hors atmosphère ($\approx 1361 \text{ W/m}^2$, modulé de $\pm 3,3\%$ par l'excentricité de l'orbite terrestre) et `airmass` la masse d'air relative (l'épaisseur d'atmosphère traversée, 1 au zénith) : deux ingrédients requis par le modèle de Perez, calculés une seule fois et passés explicitement pour éviter 192 recalculs.
- `cles` prépare les clés de groupement (mois, heure) – voir `profMH`.

Étape 2 – transposition sur les 192 plans :

```

B = np.zeros((len(ALPHAS), len(BETAS), 12, 24), np.float32)
D = np.zeros_like(B)
for i, a in enumerate(ALPHAS):
    for j, b in enumerate(BETAS):
        poa = pvlib.irradiance.get_total_irradiance(
            surface_tilt=b, surface_azimuth=a,
            solar_zenith=sp["apparent_zenith"], solar_azimuth=sp["azimuth"],
            dni=dni, ghi=ghi, dhi=dhi,
            dni_extra=dni_extra, airmass=airmass, albedo=ALBEDO, model="perez")
        direct = poa["poa_direct"].fillna(0)
        diffus = (poa["poa_sky_diffuse"] + poa["poa_ground_diffuse"]).fillna(0)
        B[i, j] = profMH(direct, cles)
        D[i, j] = profMH(diffus, cles)

```

`get_total_irradiance` est la fonction `pvlib` qui applique d'un coup les équations (2), (4) et (5) à toute la série temporelle (calcul vectorisé : chaque appel traite les 166 500 heures d'un coup ; la boucle Python ne fait que 192 tours). Elle renvoie les composantes séparées dans le plan ; on regroupe diffus ciel + réfléchi sol dans *D* (la part insensible au masquage) et on garde le direct seul dans *B* (la part qui sera annulée à l'ombre, section 4.4). Les `fillna(0)` neutralisent les NaN que Perez produit la nuit (masse d'air non définie sous l'horizon) – de l'irradiance nulle, donc.

Chaque profil est immédiatement réduit en moyenne (mois, heure) par `profMH` : on ne stocke jamais 192 séries de 166 500 points, mais 192 tableaux 12×24 . La mémoire reste constante.

Étape 3 – profils de position solaire :

```

SAZ = profMH(sp["azimuth"], cles)
SEL = profMH(sp["apparent_elevation"], cles)
return B, D, SAZ, SEL

```

Ces deux tableaux 12×24 donnent la position *moyenne* du soleil pour chaque case du jour type – exactement ce qu'il faudra, en phase tuile, pour comparer le soleil à l'horizon de chaque

pixel (équation 6 de Buffat) *sans jamais relancer de calcul astronomique*. Moyenner un azimut est en général risqué (discontinuité $360 \rightarrow 0$), mais sans danger ici : en France métropolitaine, le soleil ne passe jamais au nord pendant la journée ; la discontinuité ne concerne que des heures nocturnes, jamais utilisées (le direct y est nul).

Coût mesuré. Environ une minute par cellule, téléchargement compris (192 transpositions de 19 ans). C'est le prix payé *une fois par cellule* de $0,05^\circ$ – ensuite, tous les pixels de toutes les dalles de la cellule se servent dans la table.

6.1.4 profMH(serie, cles) : la réduction (mois, heure)

Rôle. Transformer une série temporelle quelconque en son profil moyen 12×24 (section 4.5).

```
g = serie.groupby(cles).mean()          # cles = [times.month, times.hour]
out = np.zeros((12, 24), np.float32)
for (m, h), v in g.items():
    out[int(m) - 1, int(h)] = v
return out
```

Le `groupby` de pandas range chaque valeur horaire dans sa case (mois, heure) et calcule la moyenne de chaque case : une opération standard et très optimisée. La petite boucle finale (≤ 288 tours) ne fait que recopier le résultat dans un tableau numpy dense, en décalant le mois (1–12) vers l'indice (0–11). Les cases sans données resteraient à 0 – cas qui ne se produit pas avec des séries PVGIS complètes, mais la fonction est robuste à des trous.

6.1.5 totAb(B, D) : le contrôle annuel

Rôle. Réduire des tables $(\alpha, \beta, 12, 24)$ en irradiation annuelle par (α, β) , en $\text{kWh}/\text{m}^2/\text{an}$ – l'équation (8) sommée sur l'année :

```
E = (B + D).astype(np.float64)          # W/m2 moyens par case
return (E * N_JOURS[None, None, :, None]).sum(axis=(2, 3)) / 1000.0
```

Chaque case (mois, heure) est une puissance moyenne qui s'applique 1 h par jour type : multipliée par le nombre de jours du mois (diffusion numpy sur l'axe mois : `N_JOURS[None, None, :, None]` aligne le vecteur de 12 jours sur le bon axe du tableau 4D), sommée sur mois et heures, divisée par 1000 (Wh $\rightarrow$ kWh). Le passage en `float64` évite les pertes de précision dans la somme de 288 termes stockés en `float32`.

C'est l'outil de **validation** : sur la cellule (46,30 N, 0,05 E), il donne 1346 $\text{kWh}/\text{m}^2/\text{an}$ à plat, 1582 plein sud à 30° , 915 plein nord à 30° – valeurs conformes à ce que PVGIS affiche pour la région (Poitou). Servira aussi en phase tuile pour produire la carte annuelle.

6.1.6 cheminTable et chargerTable : le stockage

Rôle. Définir où vivent les tables et comment les relire.

```
def cheminTable(lat, lon):
    return os.path.join(DOSSIER, f"table_{lat + 0.0:.2f}_{lon + 0.0:.2f}.npz")

_cache = {}
def chargerTable(lat, lon):
    la = round(round(lat / PAS) * PAS, 2)          # centre de cellule le plus proche
    lo = round(round(lon / PAS) * PAS, 2)
```



```

if (la, lo) not in _cache:
    d = np.load(cheminTable(la, lo))
    _cache[(la, lo)] = (d["B"], d["D"], d["SAZ"], d["SEL"])
return _cache[(la, lo)]

```

- Le nom encode le centre de cellule arrondi à deux décimales (`table_46.30_0.05.npz`). Le `+ 0.0` est une subtilité flottante : près du méridien de Greenwich (qui traverse la France), un arrondi peut produire le « zéro négatif » `-0.0`, qui s'afficherait `-0.00` et créerait deux noms pour la même cellule ; lui ajouter `0,0` force le zéro positif.
- `chargerTable` accepte *n'importe quel* point (typiquement le centre d'une dalle LiDAR) et retrouve la cellule météo qui le contient par arrondi au multiple de $0,05^\circ$. C'est le **point d'entrée** de la future phase tuile.
- Le cache (un dictionnaire de module) garantit une seule lecture disque par cellule et par exécution – des dalles voisines partagent la même cellule météo.

6.2 src/irradiance/irr_dep.py : le script de campagne

Ce script construit les tables pour une zone d'étude. La zone est une bbox WGS84 configurée en tête de fichier (actuellement un rectangle sur la Vienne, $7 \times 8 = 56$ cellules).

6.2.1 grilleCellules()

Génère la liste des centres de cellules couvrant la bbox : `np.arange` de `LAT_MIN` à `LAT_MAX` par pas de `0,05`, idem en longitude, produit cartésien. Le `np.round(..., 2)` corrige les erreurs d'accumulation flottante d'`arange` (qui peut produire `46,3000000000004`), pour que les noms de fichiers soient exacts.

6.2.2 construireCellule(lat, lon)

L'unité de travail : une cellule de bout en bout.

```

df = telecharger(lat, lon)
B, D, SAZ, SEL = transpAgr(df["poa_direct"], df["poa_sky_diffuse"], lat,
                           lon)
np.savez_compressed(cheminTable(lat, lon), B=B, D=D, SAZ=SAZ, SEL=SEL,
                    alphas=ALPHAS, betas=BETAS, lat=lat, lon=lon,
                    source="PVGIS-SARAH3 2005-2023, Perez")

```

Le fichier `.npz` (archive numpy compressée) contient les quatre tableaux *et* leurs axes (`alphas`, `betas`) et métadonnées (position, source) : chaque table est auto-décrite, on peut la relire dans deux ans sans ce document. Taille : ≈ 155 ko par cellule (les tableaux *B* et *D* font $24 \times 8 \times 12 \times 24$ valeurs `float32`, soit `0,44` Mo bruts, bien compressés car pleins de zéros nocturnes). La France entière ($\sim 43\,000$ cellules) tiendrait dans ~ 7 Go : le pré-calcul national est réaliste.

6.2.3 main() : la boucle robuste

```

for k, (lat, lon) in enumerate(pts, 1):
    if os.path.exists(cheminTable(lat, lon)):
        continue # deja faite : reprise gratuite
    try:
        construireCellule(lat, lon)
    except Exception as e:
        print(f"... ECHEC : {e}"); ratees.append((lat, lon))
    time.sleep(PAUSE) # politesse API (1 s)

```

Trois mécanismes pour une campagne de téléchargement fiable : (i) l’idempotence – une cellule déjà construite est sautée, on peut interrompre et relancer à volonté ; (ii) l’isolation des échecs – une erreur (réseau, API) est affichée et mémorisée mais n’arrête pas la campagne, et une simple relance retentera les manquantes ; (iii) la pause de 1 s entre requêtes, par courtoisie envers l’API publique (limite officielle : 30 requêtes/s). Coût mesuré : ≈ 1 minute par cellule, dominé par le calcul de transposition.

6.3 src/irradiance/irr_tuile.py : le pont dalle $\rightarrow$ cellule (helper)

Deux petites fonctions qui font le lien entre le monde IGN (dalles Lambert 93) et le monde météo (cellules WGS84). C’est un **helper pour les tests** sur une dalle isolée ; il ne fait pas partie de la pipeline finale (qui bouclera sur les dalles avec sa propre logique de nommage) :

```
def nomCoord(MNS_name):
    # "LHD_FXX_0495_6611_..." -> (495,
    6611)
    p = MNS_name.replace(".tif", "").split("_")
    return int(p[2]), int(p[3]) # coin NORD-ouest, en km Lambert 93

def centreWGS84(x_km, y_km):
    xc, yc = x_km * 1000 + 500, y_km * 1000 - 500 # -500 : le nom =
    coin NORD-ouest
    tr = Transformer.from_crs(2154, 4326, always_xy=True)
    lon, lat = tr.transform(xc, yc)
    return lat, lon
```

centreWGS84 place le centre de la dalle (+500 m vers l’est, –500 m vers le sud depuis le coin nord-ouest) puis le convertit en latitude/longitude avec pyproj (vérifié contre l’emprise réelle d’une dalle : écart < 1 m). C’est ainsi que la phase tuile retrouve la cellule météo de la dalle : chargerTable(*centreWGS84(*nomCoord(MNS_NAME))). Une dalle de 1 km étant bien plus petite qu’une cellule météo de $\sim 4 \times 5,5$ km, prendre la cellule du centre de dalle pour toute la dalle est une approximation tout à fait acceptable.

Le fichier irr_calcul.py (section 7) contient, lui, les fonctions de calcul réutilisées par la pipeline complète : irrPixels, masquerHorizon, agregerBatiment et irradianceTuile.

7 La phase tuile : du pixel au GeoPackage

C’est l’étape qui **croise les deux branches** et produit le fichier de sortie. Elle vit dans irr_calcul.py (le cœur de calcul réutilisable par la pipeline complète) et est appelée en dernier par main_MNTMNS.py. Elle applique l’**ombrage par l’horizon** ; le seul raffinement reporté est l’horizon des dalles voisines (section 9).

7.1 L’idée : un profil par pixel, masqué, puis intégré

L’ombrage agit *heure par heure* (le soleil n’est caché qu’à certains moments de la journée), donc un total annuel par orientation ne suffit pas : il faut le **profil** (12×24) de chaque pixel. La phase tuile suit donc exactement la phase 3 de Buffat :

1. pour chaque pixel, lire le profil direct B et diffus D de la case (α, β) la plus proche dans les tables météo ;
2. **éteindre le direct** aux cases (mois, heure) où le soleil est sous l’horizon du pixel (masquerHorizon) ;
3. **intégrer** $B_{\text{masqué}} + D$ sur l’année ($\text{kWh/m}^2/\text{an}$), puis multiplier par la surface réelle.

Un profil pèse 288 valeurs ; pour les N pixels d’une dalle on manipule donc des tableaux $(N, 12, 24)$ – quelques centaines de Mo, sans problème.

7.2 irrPixels : profil, ombrage et énergie de chaque pixel

Rôle. Construire un tableau avec **une ligne par pixel de toiture**, portant son irradiation annuelle (ombrage compris), sa surface réelle, et de quoi l'agréger ensuite.

```
masque_bat = np.where(masque_incline > 0, masque_incline, masque_plat)
toit = masque_bat > 0
p = pente[toit]; a = aspect[toit]

i = np.round(a / pas_a).astype(int) % len(alphas)    # plus proche
               orientation
j = np.clip(np.round(p / pas_b).astype(int), 0, len(betas) - 1)  # plus
               proche pente
B_pix = B[i, j]                                           # (N, 12, 24) direct
D_pix = D[i, j]                                           # (N, 12, 24) diffus + reflechi

B_pix = masquerHorizon(B_pix, horizon, SAZ, SEL)        # ombrage du direct

e_m2 = ((B_pix + D_pix) * N_JOURS[None, :, None]).sum(axis=(1, 2)) /
        1000.0
surf = 0.25 / np.cos(np.radians(p))                    # surface réelle du
               pixel (m2)

return pd.DataFrame({
    "id":        masque_bat[toit] - 1,                  # index du batiment
               dans gdf
    "energie":    e_m2 * surf,                          # kWh/an
    "surf":       surf,
    "pente":      p,
    "secteur":    np.round(a / 45).astype(int) % 8,     # secteur d'
               orientation (0=N)
    "incline":    masque_incline[toit] > 0,
})
```

Points à comprendre :

- **Fusion des deux masques.** `masque_bat` réunit toits inclinés et plats : chaque pixel de toit porte l'identifiant (+1) de son bâtiment, 0 ailleurs. Les deux masques sont exclusifs (seuil de pente), donc aucun conflit. L'ordre des pixels de `toit` est identique à celui du tableau `horizon` (même masque, même parcours en lignes), ce qui aligne les deux – une assertion le vérifie.
- **Lecture « plus proche voisin ».** Plutôt qu'une interpolation, on arrondit l'aspect au pas de la grille (15°) et la pente au sien (10°) pour tomber sur la case la plus proche des tables. Le `% len(alphas)` gère naturellement le bouclage de l'azimut (357° → case 0°). `B[i, j]` extrait alors le profil (12, 24) des N pixels d'un coup. Une interpolation bilinéaire (plus douce) est un raffinement futur (section 9).
- **Intégration (= totAb par pixel).** Chaque case (mois, heure) est une puissance moyenne valant 1 h par jour type : on multiplie par le nombre de jours du mois, on somme sur l'année, on divise par 1000 (Wh → kWh). C'est l'équation (8) appliquée pixel par pixel, après masquage du direct.
- **Surface réelle vs projetée.** Un pixel couvre $0,5 \times 0,5 = 0,25 \text{ m}^2$ au sol ; la surface réelle du pan incliné est $0,25 / \cos \beta$ (un toit à 45° a $\sqrt{2}$ fois plus de surface que son ombre au sol). C'est elle qui porte les panneaux.
- **Le secteur d'orientation.** `round(a/45) mod 8` range l'aspect dans l'un des 8 secteurs de 45° (0 = N, 2 = E, 4 = S, 6 = O) : base de la « surface par orientation » (voir plus bas).
- Tout est **vectorisé** : aucune boucle sur les pixels.

7.3 masquerHorizon : l'ombrage du direct

Rôle. Mettre à zéro le rayonnement direct des cases (mois, heure) où le soleil est caché par l'horizon local du pixel – la règle de Buffat (équation 6, section 4.4). Le diffus n'est jamais touché.

```
n_dir = horizon.shape[1]
d = np.round(SAZ / (360 / n_dir)).astype(int) % n_dir    # direction ~
    azimuth du soleil
horizon_soleil = horizon[:, d]                            # (N, 12, 24)
B_pix[SEL[None, :, :] <= horizon_soleil] = 0.0          # soleil sous l'
    horizon -> ombre
```

- **Quelle direction d'horizon ?** Pour chaque case (mois, heure), le soleil est à l'azimut $SAZ[m, h]$; on prend la direction d'horizon la plus proche (SAZ arrondi au pas de 10° , modulo 36). `horizon[:, d]` avec d de forme (12,24) produit, par indexation avancée, le tableau (N, 12, 24) donnant pour chaque pixel l'horizon dans la direction du soleil à chaque instant.
- **Le test d'ombre.** Si l'élévation du soleil $SEL[m, h]$ est inférieure ou égale à cet horizon, le pixel est à l'ombre à cet instant : son direct passe à 0 (le diffus et le réfléchi arrivent toujours).
- **La nuit est neutre.** Aux heures nocturnes le direct est déjà nul (rien à éteindre); le masquage n'a donc aucun effet parasite, même si l'azimut moyen de nuit (proche du nord) est mal défini.
- **Sur place.** On modifie `B_pix` directement (c'est déjà une copie issue de `B[i, j]`), pour éviter d'allouer un second tableau (N, 12, 24).

7.4 agregerBatiment : sommer par bâtiment

Rôle. Regrouper les pixels par bâtiment (« zonal statistics ») et joindre le résultat aux polygones BD TOPO.

```
inc = df[df.incline]                                     # pixels inclines seulement
res = pd.DataFrame({
    "irr_an_kwh":      df.groupby("id").energie.sum(),
    "surf_inclinee_m2": inc.groupby("id").surf.sum(),
    "surf_plate_m2":   df[~df.incline].groupby("id").surf.sum(),
    "pente_moy":       inc.groupby("id").pente.mean(),
    "nb_pixels":       df.groupby("id").size(),
})
for s, nom in enumerate(SECTEURS):                      # surface inclinee par
    orientation
    res[f"surf_{nom}_m2"] = inc[inc.secteur == s].groupby("id").surf.sum
        ()

res = res.fillna(0.0)
res["surf_tot_m2"]   = res.surf_inclinee_m2 + res.surf_plate_m2
res["irr_an_kwh_m2"] = res.irr_an_kwh / res.surf_tot_m2
return gdf.join(res, how="inner")                        # batiments avec >= 1 pixel
    de toit
```

- `groupby("id")` fait toutes les sommes par bâtiment sans aucune boucle Python : pandas regroupe les pixels partageant le même identifiant. L'id étant l'index du `gdf`, le `join` final réaligne tout sans risque.
- **Surface par orientation, pas azimut moyen.** On ne stocke *pas* une orientation moyenne – elle n'aurait aucun sens pour un toit à deux pans opposés (la moyenne d'un pan est et

d'un pan ouest tomberait au nord ou au sud, qu'aucun pan ne regarde). On stocke à la place la **surface inclinée dans chacun des 8 secteurs** (`surf_N_m2`, `surf_E_m2`, ...), bien plus informatif : « ce bâtiment a tant de m^2 au sud, tant à l'est ».

- **Pente moyenne sur l'inclinée seulement.** Mélanger les toits plats (pente ≈ 0) fausserait le chiffre ; `pente_moy` ne porte donc que sur les pixels inclinés.
- **Intensif recalculé à partir de l'extensif.** L'irradiation moyenne `irr_an_kwh_m2` se déduit du total d'énergie divisé par la surface totale – propriété qui rendra la fusion par région triviale (section 9) : on somme les quantités additives (énergies, surfaces) puis on redivise.
- `join(how="inner")` ne garde que les bâtiments ayant au moins un pixel de toit détecté ; ceux sans toiture exploitable (trop bas, mauvaise orientation...) sont écartés du fichier.

7.5 Du gisement à la production : le modèle PV simple

`agregerBatiment` ajoute enfin deux colonnes qui passent de l'énergie *reçue* à l'électricité *produite*, par un modèle volontairement simple (le niveau « simple » de Buffat) :

```
res["puissance_kwc"] = res.surf_tot_m2 * TAUX_COUVERTURE *
    RENDEMENT_MODULE
res["prod_an_kwh"]    = res.irr_an_kwh * TAUX_COUVERTURE *
    RENDEMENT_MODULE * PERFORMANCE_RATIO
```

Qu'est-ce qu'un kWc ? Le **kilowatt-crête** (kWc, ou kWp en anglais) est la puissance d'une installation PV mesurée dans des *conditions de test standard* (STC) : un ensoleillement de 1000 W/m^2 , une température de cellule de 25°C et un spectre solaire AM1.5, conditions fixées par la norme IEC 61215². C'est la « taille » nominale d'un système – le chiffre des fiches techniques – pas sa production réelle, qui dépend du climat. Un module de rendement η soumis à ces $1000 \text{ W/m}^2 = 1 \text{ kW/m}^2$ produit exactement η kWc par m^2 ; d'où, à couverture pleine, puissance = surface $\times \eta$.

Le facteur de production. La production annuelle est l'irradiation reçue multipliée par un facteur unique, $\tau \times \eta \times PR \approx 0,156$:

- η_{module} , **rendement du panneau** = 0,20. Valeur volontairement *prudente* : les modules silicium cristallin commerciaux atteignaient en moyenne 22,0 % fin 2024 (extrêmes 18,8–23,8 % selon le fabricant)³, mais un parc de toitures réel comporte des installations plus anciennes et moins performantes.
- PR , **performance ratio** = 0,78 : regroupe toutes les autres pertes (onduleur, échauffement des cellules, salissures, câblage, désadaptation). Valeur dans la fourchette usuelle 0,75–0,85, le « moderne » dépassant 0,80⁴.
- τ , **taux de couverture** du toit par les panneaux (1 par défaut : borne haute, toute la surface exploitable couverte).

Puissance et m^2/kWc : une conséquence, pas un paramètre. La puissance installable découle du **même** rendement (puissance = surface $\times \tau \times \eta$). La « surface par kWc » classique n'est donc *pas* une constante libre mais s'en déduit : $\text{m}^2/\text{kWc} = 1/(\eta \tau) \approx 5$ – cohérent avec la règle métier de 5 à 8 m^2/kWc ⁵ (notre 5 correspond au cas optimiste $\tau = 1$). Une seule grandeur de rendement, deux sorties cohérentes. Les trois constantes (η , PR , τ) sont en tête de `irr_calcul.py`, ajustables d'une ligne. Le modèle complet à une diode de Buffat (température

2. Définition STC : voir Réf. « STC / kWc ».

3. Fraunhofer ISE, *Photovoltaics Report*, éd. 2025 – voir Réf.

4. *Photovoltaic system performance* (synthèse) et IEA-PVPS – voir Réf.

5. Ordre de grandeur installateurs / IRENA – voir Réf.

de cellule, courbe d'onduleur) reste un raffinement futur (section 9), qui ne change le résultat que de quelques pour-cent.

7.6 irradianceTuile : l'orchestrateur

Rôle. Enchaîner les trois étapes et écrire le fichier. C'est le point d'entrée appelé par `main`.

```
def irradianceTuile(masque_incline, masque_plat, pente, aspect, gdf,
    horizon, lat, lon, out_path):
    B, D, SAZ, SEL = chargerTable(lat, lon)           # table meteo de la
    cellule
    df = irrPixels(masque_incline, masque_plat, pente, aspect,
        B, D, SAZ, SEL, horizon, ALPHAS, BETAS)
    out = agregerBatiment(df, gdf)
    out.to_file(out_path, driver="GPKG")
    return out
```

On charge la table de la cellule (cette fois *SAZ*, *SEL* servent à l'ombrage), on calcule l'irradiance par pixel (ombrage compris), on agrège par bâtiment, on écrit. L'horizon vient de `compHZ` (branche B). L'écriture du GeoPackage est une seule ligne (`to_file`) – pas besoin d'en faire une fonction à part.

7.7 Le GeoPackage de sortie

Une couche vectorielle (driver `GPKG`), **une ligne par bâtiment** ayant au moins un pixel de toiture, en Lambert 93. Colonnes :

<code>cleabs, nature, usage_1,</code>	attributs BD TOPO conservés (clé de jointure + typologie)
<code>hauteur, nombre_d_etages</code>	
<code>geometry</code>	polygone d'emprise du bâtiment
<code>irr_an_kwh</code>	irradiation annuelle totale reçue (kWh/an)
<code>irr_an_kwh_m2</code>	irradiation annuelle moyenne (kWh/m ² /an)
<code>prod_an_kwh</code>	production électrique PV estimée (kWh/an)
<code>puissance_kwc</code>	puissance installable estimée (kWc)
<code>surf_tot_m2</code>	surface de toiture exploitable totale (m ² réels)
<code>surf_inclinee_m2, surf_plate_m2</code>	dont inclinée / plate
<code>surf_N_m2 ... surf_NO_m2</code>	surface inclinée par secteur d'orientation (8)
<code>pente_moy</code>	pente moyenne des pans inclinés (degrés)
<code>nb_pixels</code>	nombre de pixels de toit (contrôle qualité)

Lire une ligne (exemple réel, dalle DIDIER). Un bâtiment de `surf_tot_m2` = 122 m² de toiture exploitable *reçoit* `irr_an_kwh` ≈ 175 000 kWh/an de soleil (soit `irr_an_kwh_m2` ≈ 1430 kWh/m²/an). On pourrait y installer `puissance_kwc` ≈ 24 kWc de panneaux (122 × 0,20), qui *produiraient* `prod_an_kwh` ≈ 27 000 kWh/an d'électricité (175 000 × 0,156). Les colonnes `surf_S_m2`, `surf_E_m2`...disent vers où sont orientés ces m² (ici surtout sud-est). **À retenir :** `irr_an_kwh` est l'énergie solaire *reçue*, `prod_an_kwh` l'électricité *produite* ; ne pas les confondre.

Validation sur la dalle DIDIER (Poitiers, 0495_6611). La pipeline complète tourne en ~ 30 s (dominée par le calcul d'horizon ; la phase tuile elle-même < 1 s). Sortie : 360 bâtiments. L'irradiation moyenne `irr_an_kwh_m2` va de 769 à 1474 kWh/m²/an (moyenne 1319) – cohérent avec PVGIS pour la région. **Effet de l'ombrage :** sans masquage la moyenne était 1379 ; avec l'horizon elle descend à 1319 (−4,4 %) et le minimum chute de 1214 à 769 (un bâtiment bas très encaissé), ce qui est physiquement attendu. Contrôles de cohérence vérifiés : `surf_tot` = inclinée

+ plate, la somme des 8 secteurs égale la surface inclinée, et `irr_an_kwh_m2 = irr_an_kwh / surf_tot`. Comme le masque ne garde que les orientations 90–270°, les secteurs N, NE, NO sont vides (attendu). La logique d’ombrage est aussi validée par un test unitaire dédié (seuil, limite incluse, choix de la bonne direction d’horizon). **Production PV** (modèle simple) : sur la dalle, 40,6 GWh/an reçus → 6,3 GWh/an produits, pour 6,1 MWc installables.

8 Conventions, unités et pièges récapitulés

Grandeur	Unité / convention	Où
Coordonnées dalles, rasters	m, Lambert 93 (EPSG 2154)	branche B
Coordonnées météo, soleil	degrés, WGS84 (EPSG 4326)	branche A
Nom de dalle IGN	km du coin nord-ouest	<code>tileBounds</code> , <code>nomCoord</code>
Azimuts (toits, soleil, horizon)	boussole : 0=N, 90=E, 180=S, 270=O	partout
Pente β	degrés, 0 = plat	<code>clip</code> , tables
Angles d’horizon	degrés (jamais de radians en sortie)	<code>compHZ</code>
Heures	UTC (jamais l’heure légale)	branche A
Irradiance	W/m ² (tables <i>B</i> , <i>D</i>)	<code>transpAgr</code>
Irradiation	kWh/m ² /an	<code>totAb</code> , sortie
Surface d’un pixel	0,25 / $\cos \beta$ m ² (réelle, pas projetée)	<code>irrPixels</code>
Secteurs d’orientation	8 de 45°, 0 =N, 2 =E, 4 =S, 6 =O	<code>agregerBatiment</code>
Nodata rasters	–9999, converti en NaN en mémoire	branche B
Identifiant bâtiment dans rasters	index GeoDataFrame +1 (0 = fond)	<code>clip</code>
Indices numpy	[<code>ligne</code> , <code>colonne</code>], ligne 0 au nord	branche B
Fichier de sortie	GeoPackage, 1 ligne / bâtiment, Lambert 93	<code>irradiancesTuile</code>

Pièges déjà rencontrés et neutralisés, à garder en tête en relisant le code : le zéro négatif dans les noms de fichiers (méridien de Greenwich), la propagation des NaN par `np.maximum`, `np.isnan` interdit sur des entiers, l’ordre (lon, lat) de pyproj, et la division par $\cos \theta_z$ au crépuscule.

9 Ce qu’il reste à faire

La phase tuile (irradiation par pixel avec ombrage, agrégation par bâtiment, GeoPackage) est **faite** (section 7). Restent les raffinements suivants.

9.1 Ombrage : ce qui est capté, ce qui ne l’est pas

L’ombrage *est* appliqué, mais l’horizon est calculé sur la **dalle seule**, dans un rayon de 100 m. Il faut distinguer nettement ce que cela traite correctement de ce que cela manque – c’est la principale question de validité de la version actuelle.

Ce qui est correctement capté. Pour tout pixel à plus de 100 m du bord de la dalle, *tous* les obstacles de son rayon de recherche sont présents dans la dalle : son horizon proche est **complet**. Cela couvre l’**auto-ombrage** (un faîtage ou un étage haut qui ombrage une partie basse du même toit) et l’**ombrage par les voisins proches** (bâtiments, arbres à moins de 100 m) – l’effet d’ombrage dominant en zone bâtie. Le carré central de 800 × 800 m, soit **64 % de la dalle**, est donc traité correctement. C’est pourquoi l’ombrage mono-dalle est déjà utile et pas un simple placeholder.

Ce qui est manqué.

1. **La bande de bord** (l’anneau de 100 m le long des 4 côtés, ~36 % de la dalle) : un rayon qui sort de la dalle « retombe » sur le pixel de bord (effet du `np.clip`) au lieu de voir

les obstacles de la dalle voisine. L’horizon y est donc *sous-estimé*, l’irradiation *sur-estimée*. **Cette perte est irréductible depuis une seule dalle** : l’information n’existe pas dans le fichier, aucune astuce ne peut la recréer. La seule vraie correction est de **charger les dalles voisines** (assemblage 3×3 , ou blocs glissants pour une région, section 5.4), à coût quasi nul : une dalle pèse 16 Mo, le 3×3 fait 144 Mo (0,016 Go/km²).

2. **Le relief lointain** (collines, montagnes au-delà de ~ 100 m) : non capté. Buffat le traite par une *seconde* méthode – l’horizon depuis le centre du bâtiment sur un MNT grossier jusqu’à 5 km, puis le maximum des deux. Ce terme est ici omis. Il est **négligeable en plaine** (cas de la Vienne) mais compte en vallée encaissée ou en montagne, où un relief peut masquer le soleil bas d’hiver plusieurs heures. Donnée mobilisable plus tard : le MNT national IGN BD ALTI à 25 m (ou le RGE ALTI rééchantillonné)⁶.
3. **Le diffus n’est pas masqué** (cf. limites) : seul le direct est éteint à l’ombre.

Que faire sans les voisines ? On ne peut pas *reconstituer* l’horizon manquant, mais on peut : (i) **marquer** les bâtiments de la bande de bord (< 100 m d’un côté) d’un drapeau « horizon incomplet » pour les signaler dans le cadastre, voire les exclure des statistiques ; (ii) porter le rayon de 100 à 500 m (la valeur de Buffat) pour mieux capter les obstacles *internes* mi-distants – au prix de $\times 5$ sur le temps d’horizon, et *sans* rien changer à la bande de bord. La correction réelle reste le chargement des voisines.

9.2 Interpolation bilinéaire (α, β)

La phase tuile lit aujourd’hui la case (α, β) **la plus proche** (plus proche voisin). Une **interpolation bilinéaire** entre les 4 nœuds voisins lisserait les sauts d’irradiation entre cases. Deux précautions : l’axe des orientations est **périodique** (interpoler entre 345° et 0° via le modulo, sans « traverser » la table), et la pente se borne à $[0, 70]$. Gain modéré (la grille $15^\circ \times 10^\circ$ est déjà fine), à faire seulement si les cartes paraissent « en escalier ».

9.3 Profils mensuels et toits plats

- **Mensuel.** La sortie est aujourd’hui annuelle ; on peut ajouter les **12 valeurs mensuelles** par bâtiment (même intégration sans sommer les mois) pour le profil saisonnier.
- **Toits plats.** Les pixels plats reçoivent actuellement l’irradiation *horizontale* (leur géométrie réelle, $\beta \approx 0$). En pratique on y poserait des panneaux **sur châssis incliné** ; il faudra leur attribuer un couple (α, β) conventionnel (plein sud à l’inclinaison optimale) au moment du modèle PV.
- **Quartiles interannuels.** Comme Buffat, en plus de la moyenne long terme, les profils des années de premier et troisième quartiles donnent une fourchette ; **profMH** s’étend facilement (grouper par année d’abord).

9.4 Modèle PV complet (raffinement)

La conversion en électricité existe déjà en version **simple** (section 7) : production = $E \times \eta_{\text{module}} \times PR$ et puissance = surface $\times \tau \times \eta$ (donc ~ 5 m² par kWc, déduit du rendement). C’est suffisant pour un premier cadastre. Reste, si l’on veut comparer des technologies ou valider sur des productions réelles, le **modèle complet** de Buffat (section 3.2) : modèle électrique à une diode (paramétrisation de Batzelis), dépendance à la température de cellule (NOCT), courbe de rendement d’onduleur, dégradation. Il ne change le résultat que de quelques pour-cent.

6. IGN, BD ALTI / RGE ALTI – voir Réf.

9.5 Passage à l'échelle : multi-dalles et fusion par région

- **Boucle multi-dalles** : transformer `main_MNTMNS.py` en traitement par lot (liste de dalles, téléchargement automatique des MNS/MNT via la géoplateforme IGN), avec parallélisation par dalle (chaque dalle est indépendante – `multiprocessing` trivial). Un `.gpkg` par dalle, comme on fait déjà un `.npz` par cellule météo.
- **Fusion par région** : un bâtiment à cheval sur plusieurs dalles a ses pixels répartis entre elles, et apparaît donc dans plusieurs `.gpkg` avec des valeurs *partielles*. On ne concatène donc pas : on `groupby("cleabs")` et on **somme les colonnes extensives** (énergies, surfaces, `nb_pixels`), on garde une copie de la géométrie et des attributs, puis on **recalcule les intensives** (`irr_an_kwh_m2` = énergie totale / surface totale). C'est exact car énergie et surface sont additives, et les dalles partitionnent l'espace (aucun double comptage de pixel).

9.6 Validation

- comparer `totAb` aux estimations PVGIS en ligne pour plusieurs cellules et orientations (fait ponctuellement : écarts cohérents) ;
- comparer les pentes/aspects extraits à des toits connus (contrôle QGIS) ;
- à terme, confronter la production simulée à des installations réelles (open data Enedis, à la manière des 500 installations de Buffat).

9.7 Limites connues et choix à réévaluer

- **Horizon sur la dalle seule** : ombrage sous-estimé sur la bande de bord (36 % de la dalle) et relief lointain ignoré – analyse détaillée section 9.1. C'est la limite principale aujourd'hui.
- **Masquage binaire du direct seulement** : le diffus n'est pas réduit par les obstacles (pas de *facteur de vue du ciel*) ; or un pixel au fond d'une cour « voit » moins de ciel et reçoit donc moins de diffus. Approximation de Buffat, conservée pour rester simple ; l'erreur est modérée car en France le direct représente la majorité de l'irradiation annuelle.
- **Position solaire moyennée pour l'ombrage** : le test d'ombre utilise l'azimut/élévation *moyens* par (mois, heure) (*SAZ*, *SEL*), pas la position instantanée. Un cas où le soleil frôle l'horizon peut être mal classé. (La transposition *B*, *D*, elle, utilise les vraies positions instantanées avant moyennage ; seul le test d'ombre est moyenné.)
- **Résolution d'horizon** 10° (36 directions) : un obstacle fin (cheminée, mât) entre deux azimuts échantillonnés peut être manqué. Buffat utilise 5°.
- **Pans nord exclus** par le filtre 90–270° d'aspect : assumé pour un outil de potentiel « rentable », à lever pour un cadastre exhaustif (les secteurs N/NE/NO de sortie sont alors toujours nuls).
- **Toits plats = irradiation horizontale** : on prend leur géométrie réelle ($\beta \approx 0$) ; en pratique les panneaux y seraient sur châssis inclinés (gain non compté), à traiter au modèle PV.
- **Lecture plus proche voisin** dans la table (α, β) (pas encore d'interpolation bilinéaire).
- **Albédo constant** (0,2) et **pas de neige** : Buffat note un biais hivernal (+28 % en décembre) attribué à la neige au sol et sur les panneaux ; impact moindre en plaine française.
- **Février = 28 jours** dans `N_JOURS` : biais < 0,1 % sur l'année, négligé.
- **Qualité BD TOPO** : les bâtiments très récents ou non cartographiés échappent à la détection ; le MNS, lui, inclut les superstructures (cheminées, panneaux existants) qui rugosifient pente et aspect.

Références

- R. Buffat, S. Grassi, M. Raubal, *A scalable method for estimating rooftop solar irradiation potential over large regions*, Applied Energy 216 (2018) 389–401. – La méthode de référence de toute la pipeline.
- R. Perez et al., *Modeling daylight availability and irradiance components from direct and global irradiance*, Solar Energy 44 (1990). – Le modèle de transposition du diffus.
- I. Reda, A. Andreas, *Solar position algorithm for solar radiation applications*, Solar Energy 76 (2004). – L’algorithme SPA.
- PVGIS 5.3, Commission européenne / JRC : https://re.jrc.ec.europa.eu/pvg_tools/fr/ (API .../api/v5_3/). – Données SARAH-3.
- pvlib python : <https://pvlib-python.readthedocs.io> – bibliothèque de référence pour le solaire (positions, transposition, iotools PVGIS).
- IGN, LiDAR HD, BD TOPO, BD ALTI et RGE ALTI : <https://geoservices.ign.fr>. – Données d’entrée et MNT national (BD ALTI 25 m) mobilisable pour l’horizon lointain.
- **SARAH-3** (CM SAF / EUMETSAT) : U. Pfeifroth et al., *SARAH-3 – satellite-based climate data records of surface solar radiation*, Earth Syst. Sci. Data 16 (2024). – Rayonnement satellite (0,05°, pas 30 min, depuis 1983) derrière PVGIS.
- **STC / kWc** : norme IEC 61215 ; synthèse [https://en.wikipedia.org/wiki/Nominal_power_\(photovoltaic\)](https://en.wikipedia.org/wiki/Nominal_power_(photovoltaic)). – Définition de la puissance-crête.
- **Performance ratio** : *Photovoltaic system performance* (Wikipedia) et IEA-PVPS Task 2. – Fourchette 0,75–0,85.
- **Rendement modules** : Fraunhofer ISE, *Photovoltaics Report* (éd. 2025), <https://www.ise.fraunhofer.de>. – Moyenne c-Si 22,0 % fin 2024.
- **Surface par kWc** : ICLEI / IRENA, *Solar PV Rooftop Systems* ; règle métier 5–8 m²/kWc.